

Prompt:

8/17/2026, 10:06:15 PM

lanjut Tahap T

Response:

8/17/2026, 10:06:16 PM

Tahap T — Portfolio-to-Order Translation & Dynamic Position Sizing

Ya. **Tahap T adalah salah satu tahap paling penting**, karena di sinilah seluruh lapisan matematika yang kita bangun sebelumnya akhirnya diterjemahkan menjadi keputusan trading konkret:

Market State → Alpha → Strategy → Capital Allocation → Position → Order

Dan saya ingin mempertahankan prinsip yang sudah kita bangun:

Optimizer tidak menentukan order secara langsung.

Optimizer menentukan *berapa banyak risk/capital yang layak diberikan kepada suatu opportunity*. Execution engine kemudian menentukan bagaimana risk tersebut diwujudkan menjadi entry, SL, TP, dan order management.

Ini penting agar arsitektur kita tidak menjadi satu model besar yang sulit diuji.

T.1. Hirarki keputusan

Kita sekarang mempunyai lima level:

Level 1: Market State → Z_t
Level 2: Strategy Allocation → $w_{s,t}$
Level 3: Trade Allocation → $R_{i,t}$
Level 4: Position Sizing → $q_{i,t}$
Level 5: Order Execution → $O_{i,t}$

Misalnya:

$$C_t = \$10,000.$$

Capital allocator mengatakan:

$$w_{strategy} = 20\%.$$

Maka:

$$C_{strategy} = \$2,000.$$

Tetapi itu **belum berarti membeli aset senilai \$2,000**.

Strategy kemudian menghasilkan sebuah trade opportunity dengan risk budget:

$$R_{trade} = \$1.$$

Barulah position-sizing engine menentukan jumlah aset.

T.2. Risk budget sebagai mata uang universal

Saya menyarankan kita menjadikan **risk** sebagai bahasa penghubung seluruh sistem.

Bukan:

"Strategi A mendapatkan \$2,000."

Tetapi:

"Strategi A mendapatkan risk budget sebesar R_A ."

Misalnya:

$$R_A = \$10/day.$$

Sebuah trade kemudian mendapatkan:

$$R_{trade} = \$1.$$

Maka secara kasar:

$$N_{trade,max} = \frac{R_A}{R_{trade}} = 10.$$

Namun jumlah aktual tetap harus memperhitungkan correlation dan portfolio risk.

T.3. Trade risk

Untuk sebuah posisi long:

P_e = entry price,

P_{SL} = stop-loss price,

q = quantity.

Gross loss jika SL terkena:

$$L_{SL} = q(P_e - P_{SL}).$$

Jika ada biaya:

$F_e, \quad F_{SL},$

dan expected slippage:

$S_e, \quad S_{SL},$

maka:

$$R_{trade} = q [(P_e - P_{SL}) + S_e + S_{SL}] + F_e + F_{SL}.$$

Untuk short, tanda harga dibalik.

T.4. Position sizing

Kita ingin:

$$R_{trade} \leq R_{budget}.$$

Sehingga:

$$q \leq \frac{R_{budget} - F_e - F_{SL}}{|P_e - P_{SL}| + S_e + S_{SL}}.$$

Jadi:

$$q_{risk} = \frac{R_{budget} - F}{D_{SL} + S}$$

dengan:

$$D_{SL} = |P_e - P_{SL}|.$$

Ini merupakan **position sizing engine paling dasar**.

T.5. Tetapi kita tidak boleh berhenti di sini

Karena:

$$q_{risk}$$

belum mempertimbangkan:

- liquidity,
- market impact,
- portfolio exposure,
- correlation,
- factor exposure,
- leverage,
- confidence,
- regime.

Maka quantity final harus:

$$q_{final} = \min(q_{risk}, q_{liquidity}, q_{portfolio}, q_{leverage}, q_{capacity}).$$

T.6. Stop-loss tidak boleh menentukan risk budget

Ini prinsip penting.

Urutannya bukan:

$$SL \rightarrow Risk.$$

Tetapi:

$$Risk \rightarrow SL \rightarrow PositionSize.$$

Mengapa?

Karena kita terlebih dahulu menentukan:

"Berapa kerugian maksimum yang bersedia diterima?"

Kemudian menentukan struktur trade yang masuk akal.

T.7. Menentukan Stop Loss

SL sebaiknya bukan angka arbitrer.

Kita dapat menggunakan kombinasi:

Volatility

$$D_{SL} = k\sigma_{ATR}.$$

Misalnya:

$$D_{SL} = 2ATR.$$

Market structure

$$P_{SL} = P_{swing\ low} - \epsilon.$$

Statistical boundary

Misalnya:

$$P_{SL} = P_e - z_\alpha \sigma_h.$$

T.8. Model SL gabungan

Misalnya kita memiliki tiga estimasi:

$$D_{vol},$$

$$D_{structure},$$

$$D_{stat}.$$

Kita dapat menggunakan:

$$D_{SL} = \max(D_{vol}, D_{structure}, D_{stat}).$$

Tujuannya agar SL tidak terlalu dekat sehingga terkena noise.

T.9. Tetapi ada trade-off

SL semakin jauh:

$$D_{SL} \uparrow$$

maka:

$$q_{risk} \downarrow .$$

Karena:

$$q_{risk} \propto \frac{1}{D_{SL}}.$$

Jadi volatility tinggi secara otomatis menyebabkan:

$$position\ size \downarrow .$$

Ini jauh lebih masuk akal daripada selalu membeli jumlah aset yang sama.

T.10. Take Profit

Sekarang kita menentukan TP.

Definisikan:

$$D_{TP} = |P_{TP} - P_e|.$$

Risk distance:

$$D_{SL} = |P_e - P_{SL}|.$$

Risk-reward ratio:

$$RR = \frac{D_{TP}}{D_{SL}}.$$

T.11. Tetapi TP seharusnya berasal dari expected distribution

Ini lebih menarik.

Misalkan model kita memperkirakan:

$$R_{future} \sim D(\theta).$$

Kita ingin memilih TP:

$$P_{TP}$$

berdasarkan distribusi return, bukan sekadar:

$$TP = 2 \times SL.$$

T.12. Expected value

Misalkan:

$$p_w = P(\text{TP hit}),$$

$$p_l = P(\text{SL hit}),$$

dan:

$$p_n = 1 - p_w - p_l.$$

Dengan payoff:

$$G = TP - \text{entry},$$

$$L = \text{entry} - SL.$$

Expected PnL:

$$E[PnL] = p_w G - p_l L + E[PnL|\text{neutral}]p_n.$$

Jika neutral diabaikan:

$$E[PnL] = p_w G - p_l L.$$

T.13. Break-even probability

Jika:

$$E[PnL] > 0,$$

maka:

$$p_w G > p_l L.$$

Jika hanya dua outcome:

$$p_l = 1 - p_w.$$

Maka:

$$p_w G > (1 - p_w)L.$$

Sehingga:

$$p_w(G + L) > L.$$

Jadi:

$$p_w > \frac{L}{G + L}.$$

Untuk:

$$RR = \frac{G}{L},$$

maka:

$$p_w > \frac{1}{1 + RR}.$$

T.14. Contoh

Jika:

$$RR = 2,$$

maka:

$$p_w > \frac{1}{3}.$$

Jadi win rate hanya:

$$33.3\%$$

sudah cukup secara teoritis sebelum biaya dan slippage.

Tetapi tentu model harus menghasilkan probability yang *out-of-sample calibrated*.

T.15. Expected utility

Kita bisa menjadi lebih sophisticated.

Daripada:

$$\max E[PnL],$$

gunakan utility:

$$U(W) = E[W] - \frac{\lambda}{2} Var(W).$$

Untuk trade:

$$U = E[R] - \frac{\lambda}{2} Var(R).$$

Maka TP dipilih untuk:

$$TP^* = \arg \max_{TP} U(TP).$$

T.16. Ini membuat TP menjadi optimization problem

Bukan:

$$TP = 2 \times SL$$

tetapi:

$$TP^* = \arg \max_{TP} [E[PnL(TP)] - \frac{\lambda}{2} Var(PnL(TP)) - Cost(TP)].$$

Ini jauh lebih sesuai dengan sistem kita.

T.17. Sekarang masalah yang kamu tanyakan sebelumnya

Jika posisi sudah floating profit tinggi, apakah TP dinaikkan?

Jawabannya:

bisa, tetapi bukan karena profitnya tinggi.

Kita harus bertanya:

Apakah conditional expected return masih positif?

T.18. Conditional expected return

Misalkan posisi sudah menghasilkan:

$$P_t > P_e.$$

State sekarang:

$$S_t.$$

Model menghitung:

$$E[R_{t:t+H}|S_t].$$

Jika:

$$E[R_{future}|S_t] > 0,$$

maka posisi masih memiliki expected edge.

Jika:

$$E[R_{future}|S_t] \approx 0,$$

maka tidak ada alasan kuat untuk mempertahankannya.

Jika:

$$E[R_{future}|S_t] < 0,$$

maka exit mungkin lebih rasional.

T.19. Ini menghasilkan prinsip penting

Jangan menggunakan:

floating profit

sebagai signal utama.

Gunakan:

future conditional edge.

Floating profit hanya merupakan bagian dari state.

T.20. State posisi

Kita definisikan:

$$X_t = [P_t, P_e, SL_t, TP_t, MFE_t, MAE_t, \tau_t, \sigma_t, \mu_t, Z_t, L_t].$$

Dengan:

- MFE : maximum favorable excursion,
 - MAE : maximum adverse excursion,
 - τ_t : holding time,
 - Z_t : regime,
 - L_t : liquidity.
-

T.21. Dynamic Stop Loss

Kita definisikan:

$$SL_t = f(X_t).$$

Tetapi kita harus menghindari:

$$SL_t$$

yang terus bergerak mengikuti harga tanpa alasan statistik.

T.22. Break-even transition

Misalnya:

$$R_t = \frac{P_t - P_e}{P_e - P_{SL,0}}.$$

Jika:

$$R_t \geq r_{BE},$$

maka SL dapat dipindahkan ke:

$$SL_t = P_e + c$$

dengan c memperhitungkan fees/slippage.

T.23. Contoh

Initial:

$$P_e = 100.$$

SL:

$$95.$$

Risk:

$$5.$$

Jika harga:

$$P_t = 105,$$

maka:

$$R_t = 1R.$$

Kita dapat memindahkan:

$$SL = 100$$

atau sedikit di atas:

$$SL = 100 + \epsilon.$$

T.24. Tetapi BEP terlalu agresif

Masalah:

$$SL = entry$$

membuat trade sangat mudah keluar akibat noise.

Jadi kita dapat menggunakan:

$$SL_t = P_e + \alpha ATR_t.$$

Misalnya:

$$\alpha = 0.2.$$

Untuk long:

$$SL_t = P_e + 0.2 ATR_t.$$

T.25. Trailing stop berbasis volatility

Alternatif:

$$SL_t = P_t - k\sigma_t$$

untuk long.

Dan:

$$SL_t = P_t + k\sigma_t$$

untuk short.

Tetapi:

$$k$$

harus dikalibrasi menggunakan data OOS.

T.26. Bahkan lebih baik: trailing stop berdasarkan distribution

Misalkan model menghasilkan:

$$P(R_{t+h} < -\delta | S_t).$$

Kita ingin SL berada pada quantile tertentu:

$$P(R_{future} < -D_{SL} | S_t) = \alpha.$$

Sehingga:

$$D_{SL} = \text{Quantile}_\alpha(-Loss_{future} | S_t).$$

Ini menghasilkan **probabilistic stop-loss**.

T.27. TP dinamis

Demikian pula:

$$TP_t = f(P_t, S_t, \hat{\mu}_t, \hat{\sigma}_t, Z_t).$$

Kita dapat menentukan:

$$TP_t^* = \arg \max_{TP} E[U | S_t].$$

T.28. Tetapi TP tidak boleh terus dinaikkan

Misalkan:

$$TP_0 = 110.$$

Harga:

$$P_t = 108.$$

Model baru mengatakan target optimal:

$$TP^* = 109.$$

Maka kita **tidak** menaikkan TP menjadi 115 hanya karena harga sedang profit.

Kita justru menurunkannya.

Jadi:

$$TP_t = \text{model-driven,}$$

bukan:

profit-driven.

T.29. Ini memberi kita tiga aksi

Pada setiap evaluation period:

$$a_t \in \{HOLD, MODIFY, EXIT\}.$$

HOLD

Jika:

$$E[R_{future}|S_t] > threshold.$$

MODIFY

Jika:

$$E[R_{future}|S_t] > 0$$

tetapi optimal:

$$SL_t^* \ni SL_{t-1}$$

atau:

$$TP_t^* \ni TP_{t-1}.$$

EXIT

Jika:

$$E[R_{future}|S_t] \leq 0.$$

T.30. Ini sebenarnya adalah optimal stopping problem

Secara matematis:

$$\tau^* = \inf \{t : V(S_t) \leq E_{exit}(S_t)\}.$$

Dengan:

$$V(S_t)$$

sebagai continuation value.

T.31. Continuation value

Kita dapat mendefinisikan:

$$V_t = E[U(W_{t+\Delta}) | S_t, \text{continue}].$$

Exit value:

$$E_t^{exit} = U(W_t - C_{exit}).$$

Maka:

$$\text{Continue} \Leftrightarrow V_t > E_t^{exit}.$$

Ini jauh lebih fundamental daripada aturan:

"Kalau floating profit, geser SL."

T.32. Holding time juga penting

Misalkan trade sudah berjalan:

$$\tau = 3 \text{ jam.}$$

Model awal memprediksi edge dalam:

$$H = 2 \text{ jam.}$$

Jika setelah 3 jam target belum tercapai:

$$P(TP|\tau)$$

mungkin sudah turun.

Ini disebut **time decay of alpha**.

T.33. Hazard function

Kita dapat mendefinisikan:

$$h_{TP}(t) = \lim_{\Delta t \rightarrow 0} \frac{P(t \leq T_{TP} < t + \Delta t \mid T_{TP} \geq t)}{\Delta t}.$$

Jika:

$$h_{TP}(t)$$

menurun secara signifikan, continuation probability juga turun.

T.34. Maximum holding time

Kita dapat menentukan:

$$\tau_{max}$$

dengan:

$$E[R_{future}|S_t, \tau] < threshold.$$

Maka:

$$\tau > \tau_{max} \Rightarrow EXIT.$$

T.35. Sekarang portfolio interaction

Ini sangat penting.

Misalkan kita memiliki:

$$Trade_A = BTC$$

dan:

$$Trade_B = ETH.$$

Keduanya floating profit.

Trade A memiliki:

$$+\$3.$$

Trade B:

$$+\$2.$$

Tetapi market suddenly masuk regime:

$$HighVol.$$

Correlation:

$$\rho_{BTC,ETH} \uparrow .$$

Portfolio risk meningkat.

Maka mungkin kita tidak mempertahankan keduanya.

T.36. Portfolio-level position management

Kita harus menghitung:

$$\sigma_{portfolio,t} = \sqrt{\mathbf{q}_t^T \Sigma_t \mathbf{q}_t}.$$

Jika:

$$\sigma_{portfolio,t} > B_t,$$

maka kita harus melakukan **risk reduction**.

T.37. Trade selection saat risk budget penuh

Misalkan terdapat tiga posisi:

$$A, B, C.$$

Risk contributions:

$$RC_A = 40\%,$$

$$RC_B = 35\%,$$

$$RC_C = 25\%.$$

Tetapi expected future alpha:

$$\alpha_A = 0.1,$$

$$\alpha_B = 0.5,$$

$$\alpha_C = 0.3.$$

Jika perlu mengurangi risk, kita tidak otomatis menutup posisi yang paling profit.

Kita mencari:

$$\text{lowest marginal utility of risk.}$$

T.38. Marginal utility of risk

Definisikan:

$$U_p = E[R_p] - \lambda \sigma_p^2.$$

Marginal contribution:

$$\frac{\partial U_p}{\partial w_i} = \mu_i - 2\lambda(\Sigma w)_i.$$

Jika:

$$\frac{\partial U_p}{\partial w_i} < 0,$$

strategy tersebut sedang memberikan kontribusi marginal negatif terhadap objective.

T.39. Ini menghasilkan exit ranking

Kita dapat menghitung:

$$Score_i = \frac{Marginal\ Alpha_i}{Marginal\ Risk_i}.$$

Kemudian posisi dengan:

$$Score_i$$

paling rendah dikurangi terlebih dahulu.

T.40. Ini jauh lebih baik daripada FIFO

Bukan:

posisi tertua ditutup.

Bukan:

posisi yang profit paling kecil ditutup.

Bukan:

posisi yang profit paling besar ditutup.

Tetapi:

posisi dengan marginal risk – adjusted value terendah

yang dikurangi.

T.41. Liquidity constraint

Sekarang kita masuk ke masalah nyata.

Kita tidak boleh menganggap:

q

bisa berapa pun.

Misalkan:

ADV = average daily volume.

Kita dapat membatasi:

$$q \leq \eta ADV.$$

Dengan:

η

misalnya beberapa persen atau jauh lebih kecil tergantung market.

T.42. Market impact

Approximation sederhana:

$$Impact(q) = Y \sigma \left(\frac{q}{V} \right)^\alpha.$$

Dengan:

- Y : impact coefficient,
- σ : volatility,
- V : volume,
- α : scaling exponent.

Maka effective execution price menjadi:

$$P_{exec} = P_{mid} + Impact(q).$$

T.43. Sekarang position size menjadi fixed point

Karena:

$$q$$

menentukan:

$$Impact(q),$$

dan impact mempengaruhi:

$$Risk(q).$$

Maka:

$$q = \frac{R_{budget}}{D_{SL} + Impact(q)}.$$

Jadi:

$$q^* = \arg \max_q U(q)$$

dengan impact diperhitungkan.

T.44. Untuk market sangat liquid

Jika:

$$\text{Impact}(q) \approx 0,$$

maka:

$$q^* \approx q_{\text{risk}}.$$

Tetapi untuk:

- small-cap stocks,
- mid-cap crypto,
- altcoins,
- thin order books,

impact dapat menjadi faktor dominan.

T.45. Order book

Kita sekarang masuk ke order execution.

Order book:

$$B_t = \{(p_i^b, v_i^b)\}$$

dan:

$$A_t = \{(p_j^a, v_j^a)\}.$$

Bid:

$$B_t = \max p_i^b.$$

Ask:

$$A_t = \min p_j^a.$$

Spread:

$$S_t = A_t - B_t.$$

T.46. Mid price

$$M_t = \frac{A_t + B_t}{2}.$$

Microprice dapat menggunakan imbalance.

Order book imbalance:

$$I_t = \frac{V_t^b - V_t^a}{V_t^b + V_t^a}.$$

T.47. Order book sebagai short-term alpha

Misalnya:

$$I_t > 0.$$

Artinya bid-side depth relatif besar.

Model dapat menganggap:

$$P(\Delta P > 0 | I_t)$$

meningkat.

Tetapi:

$$I_t$$

tidak boleh dianggap sebagai signal independen; ia harus masuk ke model state kita.

T.48. Entry decision

Sekarang opportunity score:

$$A_t = f(\mu_t, \sigma_t, I_t, spread_t, liquidity_t, regime_t).$$



Trade hanya dibuat jika:

$$A_t > A_{min}.$$

T.49. Expected edge setelah transaction cost

Ini sangat penting.

Model tidak boleh menggunakan:

$$E[R].$$

Tetapi:

$$E[R_{net}] = E[R_{gross}] - E[fee] - E[spread] - E[slippage] - E[impact].$$

Trade hanya valid jika:

$$E[R_{net}] > 0.$$

T.50. Bahkan harus melewati hurdle rate

Karena estimation error:

$$E[R_{net}] > H_t.$$

Hurdle:

$$H_t = k\sigma_{\hat{\mu}} + Cost.$$

Sehingga:

$$\hat{\mu}_t > Cost_t + kSE(\hat{\mu}_t).$$

Ini membuat model lebih konservatif.

T.51. Pending order

Kamu sebelumnya menginginkan:

order diletakkan tetapi tidak langsung dieksekusi.

Maka kita memiliki:

$$P_{limit}.$$

Order valid selama:

$$t \leq t_{expiry}.$$

Jika belum terisi:

$$Order \rightarrow Cancel$$

ketika:

$$P(fill|S_t)$$

turun terlalu rendah.

T.52. Fill probability

Kita dapat memodelkan:

$$P_{fill} = f(distance, queue, volume, volatility, orderbook).$$

Misalnya:

$$P_{fill} = \sigma(\theta_0 + \theta_1 d + \theta_2 I + \theta_3 V).$$

Dengan sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

T.53. Expected value pending order

Trade limit order bukan hanya:

$$E[PNL|trade].$$

Tetapi:

$$EV_{limit} = P_{fill}EV_{trade} - Cost_{waiting} - OpportunityCost.$$

Jadi order hanya diletakkan jika:

$$EV_{limit} > 0.$$

T.54. Ini sangat penting

Karena limit order memiliki:

Adverse Selection.

Jika order hanya filled ketika harga bergerak melawan kita, maka:

$$P_{fill}$$

tidak independen terhadap future return.

Sehingga:

$$P(fill|future\ loss) \neq P(fill).$$

Ini harus dimodelkan.

T.55. Kita sekarang memiliki lifecycle lengkap





T.56. Dynamic management loop

Setiap:

$$\Delta t$$

kita menghitung:

$$S_t.$$

Kemudian:

$$\pi_t = \pi(S_t)$$

menghasilkan action:

$$a_t.$$

Secara formal:

$$a_t = \arg \max_a E[U_{t:H} \mid S_t, a].$$

T.57. Action space

Untuk posisi yang sudah aktif:

$$a_t \in \{EXIT, HOLD, MOVE_SL, MOVE_TP, PARTIAL_EXIT, ADD, REDUCE\}.$$

Saya justru menyarankan **ADD** diperlakukan sangat hati-hati.

Jangan melakukan averaging down hanya karena posisi loss.

T.58. Averaging down

Misalnya:

$$P_t < P_e.$$

Kita tidak mengatakan:

"Harga turun, beli lagi supaya average price turun."

Sebaliknya:

$$EV_{new} = EV(\text{portfolio} + \text{new position}).$$

ADD hanya dilakukan jika:

$$\boxed{\Delta U > 0}$$

dan:

$$Risk_{\text{portfolio}, new} \leq B_t.$$

T.59. Floating loss

Sekarang menjawab salah satu pertanyaan awalmu:

Jika floating loss, biarkan saja?

Tidak secara otomatis.

Floating loss bukan alasan untuk exit.

Tetapi juga bukan alasan untuk hold.

Kita evaluasi:

$$V_{continue}$$

VS:

$$V_{exit}.$$

T.60. Decision matrix

Secara sederhana:

Kondisi	Action
Loss + alpha masih kuat	HOLD
Loss + alpha melemah	REDUCE
Loss + alpha negatif	EXIT
Profit + alpha kuat	HOLD / TRAIL
Profit + alpha melemah	Tighten / Partial
Profit + alpha negatif	EXIT
Portfolio risk terlalu tinggi	REDUCE
Liquidity memburuk	REDUCE / EXIT

Jadi:

PnL bukan decision variable tunggal.

T.61. Mark-to-market bias

Ada bahaya psikologis maupun matematis:

$$P_t - P_e.$$

Informasi tersebut bisa membuat model terlalu terikat pada entry price.

Padahal setelah trade masuk, pertanyaan optimal adalah:

Jika **sekarang saya tidak memiliki posisi**, apakah saya masih ingin membuka posisi ini?

Ini disebut **decision from current state**, bukan sunk-cost reasoning.

T.62. Counterfactual decision

Kita dapat menghitung:

$$EV_{enter}(S_t)$$

seandainya kita belum memiliki posisi.

Bandingkan dengan:

$$EV_{hold}(S_t).$$

Jika:

$$EV_{enter} < 0,$$

maka posisi seharusnya tidak dipertahankan kecuali ada alasan exit-cost yang membuat continuation masih optimal.

T.63. Ini memberi kita prinsip yang sangat kuat

Every evaluation period : pretend the position does not exist.

Kemudian tanyakan:

Would the model enter this position now?

Jika tidak:

reduce/exit

menjadi kandidat kuat.

T.64. Namun kita masih harus memperhitungkan exit cost

Jika posisi ditutup sekarang:

$$C_{exit}.$$

Continuation value:

$$V_{hold}.$$

Maka:

$$\boxed{Exit \Leftrightarrow V_{exit} > V_{hold}.$$

Ini menghindari churn.

T.65. Dynamic risk budget

Risk budget juga harus berubah.

Misalkan:

$$DD_t$$

adalah drawdown.

Kita dapat mendefinisikan:

$$B_t = B_0 g(DD_t).$$

Misalnya:

$$g(DD) = \begin{cases} 1, & DD < D_1 \\ 0.75, & D_1 \leq DD < D_2 \\ 0.5, & D_2 \leq DD < D_3 \\ 0, & DD \geq DD_3. \end{cases}$$

Ini adalah **drawdown control layer**.

T.66. Tetapi kita dapat membuatnya continuous

Misalnya:

$$B_t = B_0 \exp(-\kappa DD_t).$$

Ketika drawdown meningkat:

$$DD_t \uparrow$$

maka:

$$B_t \downarrow .$$

T.67. Ini membuat sistem self-regulating

Saat performa memburuk:

$$Risk \downarrow .$$

Saat kondisi membaik:

$$Risk \uparrow .$$

Tetapi tetap dibatasi:

$$B_{min} \leq B_t \leq B_{max}.$$

T.68. Sekarang kita bisa menggabungkan semuanya

Trade risk:

$$R_{trade}.$$

Strategy risk:

$$R_{strategy}.$$

Portfolio risk:

$$R_{portfolio}.$$

Market risk:

$$R_{market}.$$

Kita ingin:

$$R_{trade} \leq R_{strategy} \leq R_{portfolio} \leq R_{market}.$$

Ini bukan inequality matematis literal dalam setiap implementasi, tetapi merupakan **hierarki risk budget**.

T.69. Final position sizing equation

Secara konseptual saya ingin kita menggunakan:

$$q_t^* = \min [q_{risk}, q_{portfolio}, q_{liquidity}, q_{leverage}, q_{capacity}]$$

dengan:

$$q_{risk} = \frac{R_{trade,t} - C_t}{D_{SL,t} + S_t}.$$

Tetapi risk budget-nya sendiri dinamis:

$$R_{trade,t} = R_0 \cdot c_t \cdot g(Z_t) \cdot g(DD_t) \cdot g(\sigma_t)$$

dengan c_t sebagai confidence.

T.70. Ini menghubungkan hampir seluruh tahap sebelumnya

Perhatikan:

$$R_{trade,t} = R_0 \times Confidence \times Regime \times Drawdown \times Volatility.$$

Jadi:

Model confidence tinggi

$$c_t \uparrow \Rightarrow R_t \uparrow .$$

Regime buruk

$$g(Z_t) \downarrow \Rightarrow R_t \downarrow .$$

Drawdown tinggi

$$g(DD_t) \downarrow \Rightarrow R_t \downarrow .$$

Volatility tinggi

$$g(\sigma_t) \downarrow \Rightarrow R_t \downarrow .$$

T.71. Tetapi jangan double-count risk

Ini peringatan penting.

Jika:

$$\sigma_t$$

sudah masuk ke:

$$\Sigma_t$$

dan kita juga memasukkan:

$$g(\sigma_t)$$

terlalu agresif, kita bisa melakukan:

double risk scaling.

Jadi setiap adjustment harus memiliki definisi yang jelas.

T.72. Architecture final Tahap T

Saya ingin engine kita nanti dipisahkan menjadi:

```
T1  Signal Validation
T2  Alpha Estimation
T3  Confidence
T4  Risk Budget
T5  SL Engine
T6  TP Engine
T7  Position Sizing
T8  Portfolio Constraint
T9  Liquidity / Impact
T10 Order Placement
T11 Fill Management
T12 Position Management
T13 Dynamic SL/TP
T14 Exit Engine
T15 Risk Shutdown
```

Masing-masing bisa diuji secara independen.

T.73. Pseudocode konseptual

```
for every evaluation_time t:

    market_state = estimate_state(t)

    alpha = estimate_alpha(market_state)

    confidence = estimate_confidence(alpha)

    strategy_weight = allocate_capital(
        alpha,
        covariance,
        factors,
        confidence
    )

    for each candidate trade:

        expected_return = estimate_expected_return()

        if expected_return <= transaction_cost + hurdle:
            continue

        risk_budget = compute_dynamic_risk_budget(
            strategy_weight,
            confidence,
            volatility,
            regime,
            drawdown
        )

        stop_loss = estimate_SL(
            volatility,
            market_structure,
            distribution
        )

        position_size = solve_position_size(
            risk_budget,
            stop_loss,
            slippage,
```

```
        liquidity
    )

    if portfolio_risk_after_trade > risk_limit:
        reduce_size()

    take_profit = optimize_TP(
        expected_return_distribution,
        stop_loss,
        utility
    )

    order = construct_order(
        entry,
        position_size,
        stop_loss,
        take_profit
    )

    execute_or_place(order)
```

T.74. Dan posisi aktif

```
for every active_position:

    state = update_position_state()

    continuation_value = estimate_hold_value(state)

    exit_value = estimate_exit_value(state)

    if portfolio_risk > limit:
        reduce_position()

    elif continuation_value < exit_value:
        exit_position()

    else:

        new_SL = optimize_SL(state)
        new_TP = optimize_TP(state)

        if new_SL improves expected utility:
            modify_SL(new_SL)

        if new_TP improves expected utility:
            modify_TP(new_TP)
```

T.75. Satu hal yang sengaja belum kita lakukan

Kita **belum** membuat machine learning model yang sangat kompleks.

Ini disengaja.

Kita ingin terlebih dahulu memastikan:

Decision Theory

benar.

Barulah model seperti:

- XGBoost,

- LightGBM,
- neural network,
- transformer,
- temporal model,

digunakan untuk memperkirakan:

$$P(R_{future}|S_t)$$

atau:

$$E[R_{future}|S_t].$$

Model ML hanyalah **estimator**.

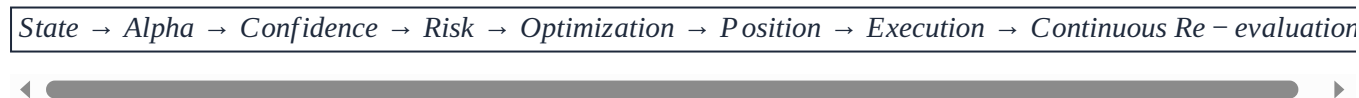
Optimizer dan risk engine tetap menjadi lapisan keputusan.

T.76. Ini membuat sistem kita berbeda dari bot trading biasa

Bot biasa:

$$Signal \rightarrow Buy/Sell.$$

Sistem kita:



Dan yang lebih penting:

Tidak ada keputusan trading yang berdiri sendiri.

T.77. Hubungan dengan TSP, MaxCut, dan portfolio optimization

Sekarang kita bisa melihat di mana problem combinatorial sebenarnya muncul.

Portfolio allocation

QUBO/QAOA

Strategy selection

QUBO/QAOA

Execution routing

Misalnya memilih venue:

QUBO

Order scheduling

Memilih urutan eksekusi:

TSP – like

Pair / asset selection

Memaksimalkan diversity:

MaxCut – like

Capital allocation

QUBO

Jadi kita tidak memaksakan satu algoritma quantum ke semua hal.

T.78. Contoh MaxCut dalam sistem kita

Misalkan kita memiliki:

10

strategi.

Kita ingin membagi strategi menjadi dua kelompok:

$Cluster_A$

dan:

$$Cluster_B.$$

Kita ingin strategi yang highly correlated berada di kelompok berbeda atau setidaknya mendapatkan penalti.

Graph:

$$G = (V, E).$$

Edge weight:

$$w_{ij} = |\rho_{ij}|.$$

MaxCut:

$$\max \sum_{(i,j) \in E} w_{ij} x_i (1 - x_j).$$

Ini dapat digunakan untuk mencari diversification structure.

T.79. TSP-like execution

Misalkan kita perlu mengeksekusi:

$$N$$

order pada beberapa venue.

Kita ingin meminimalkan:

$$ExecutionCost + Slippage + Latency + MarketImpact.$$

Jika urutan eksekusi penting, problem dapat memiliki struktur:

$$TSP/routing.$$

T.80. Jadi akhirnya kita memiliki "optimization toolbox"

Bukan:

QAOA trading bot.

Tetapi:

Quantitative Trading Optimization System

dengan toolbox:

Convex Optimization
Quadratic Programming
QUBO
Ising
QAOA
MaxCut
TSP
Dynamic Programming
Statistical Decision Theory

T.81. Tahap T dalam satu persamaan besar

Secara konseptual, seluruh keputusan order dapat dirangkum:

$$O_t^* = \arg \max_{O_t} E [U(W_{t:t+H}) \mid S_t, O_t]$$

subject to:

$$\begin{aligned} R_{trade} &\leq B_{trade,t}, \\ R_{portfolio} &\leq B_{portfolio,t}, \\ FactorExposure &\leq F_{max}, \\ Leverage &\leq L_{max}, \\ Impact &\leq I_{max}, \\ Liquidity &\geq L_{min}, \\ PositionSize &\leq Capacity, \\ ExecutionCost &\leq C_{max}. \end{aligned}$$

Inilah sebenarnya tujuan akhir dari seluruh sistem.

T.82. Dan sekarang kita memiliki loop tertutup

Sebelumnya sistem kita terlihat seperti:

Market → Signal → Order.

Sekarang:

Market → State → Optimization → Order → Position → New State → Optimization → ...

Jadi:

closed-loop adaptive trading system.

T.83. Posisi Tahap T dalam keseluruhan proyek

```
A Market & Data
B Return / Factor Modeling
C Alpha Construction
D Market Regime
E Risk Modeling
F Strategy Construction
G Portfolio / QUBO
H Quantum Mapping
I QAOA / Optimization
J Backtesting
K Execution Modeling
L Transaction Cost
M Robustness
N Walk Forward
O Stress Testing
P Statistical Validation
Q Strategy Selection
R Strategy Validation
S Strategy Ensemble & Capital Allocation
T Portfolio → Order + Dynamic Position Management
  |
  ▼
  U
```

Dan menurut saya **Tahap U** nanti harus menjadi tahap yang sangat berbeda dari **T**: bukan lagi sekadar bagaimana menentukan order, tetapi bagaimana kita membangun **execution/routing engine yang**

realistis terhadap order book, queue position, latency, slippage, partial fills, maker/taker fee, cancellation, dan adverse selection.

Itu akan menjadi lapisan yang sangat penting jika tujuan akhirnya adalah membuat sistem yang benar-benar mendekati paradigma **quantitative trading infrastructure**, bukan sekadar backtest strategy.